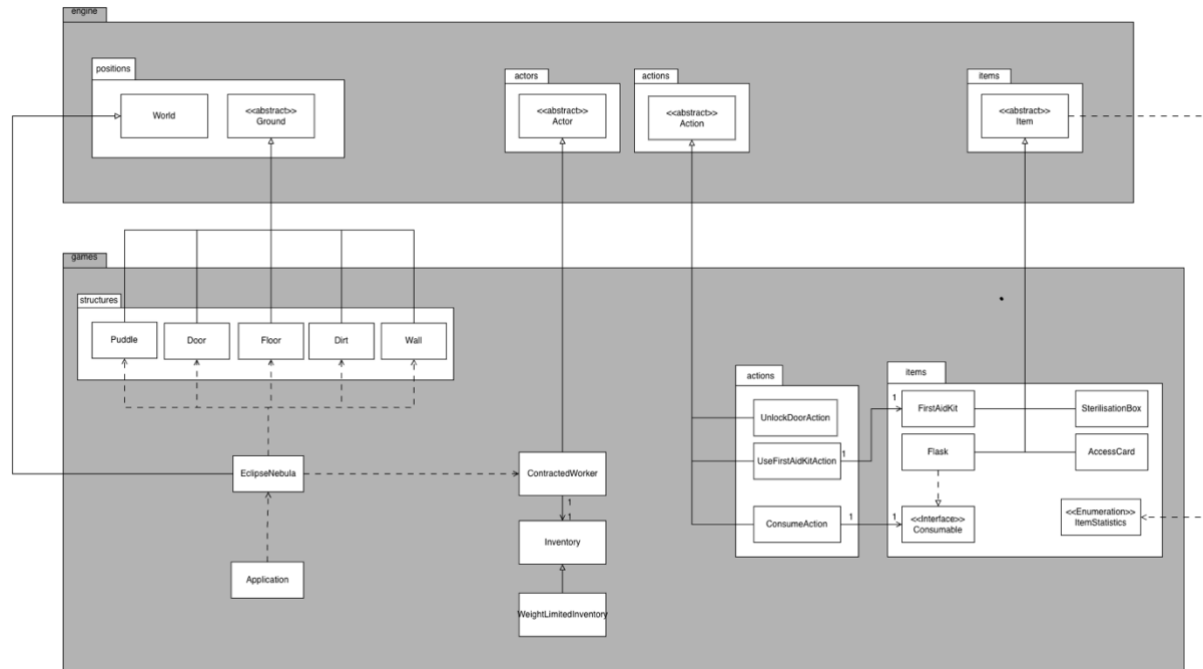


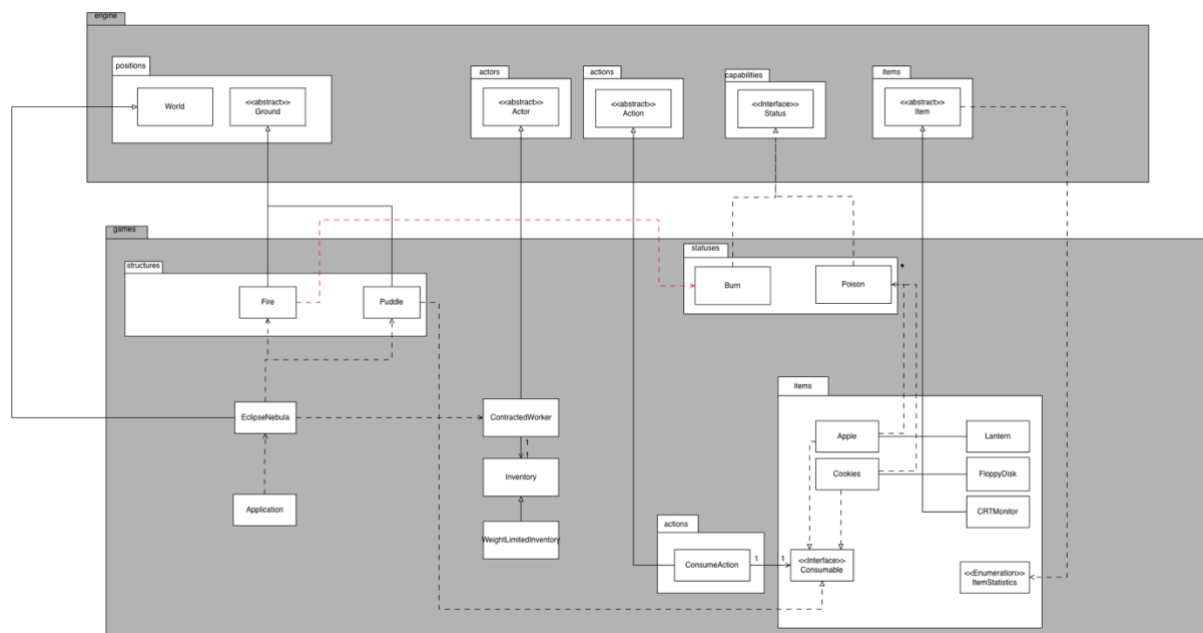
FIT2099 Assignment 1 Design Documentation

Design Diagrams

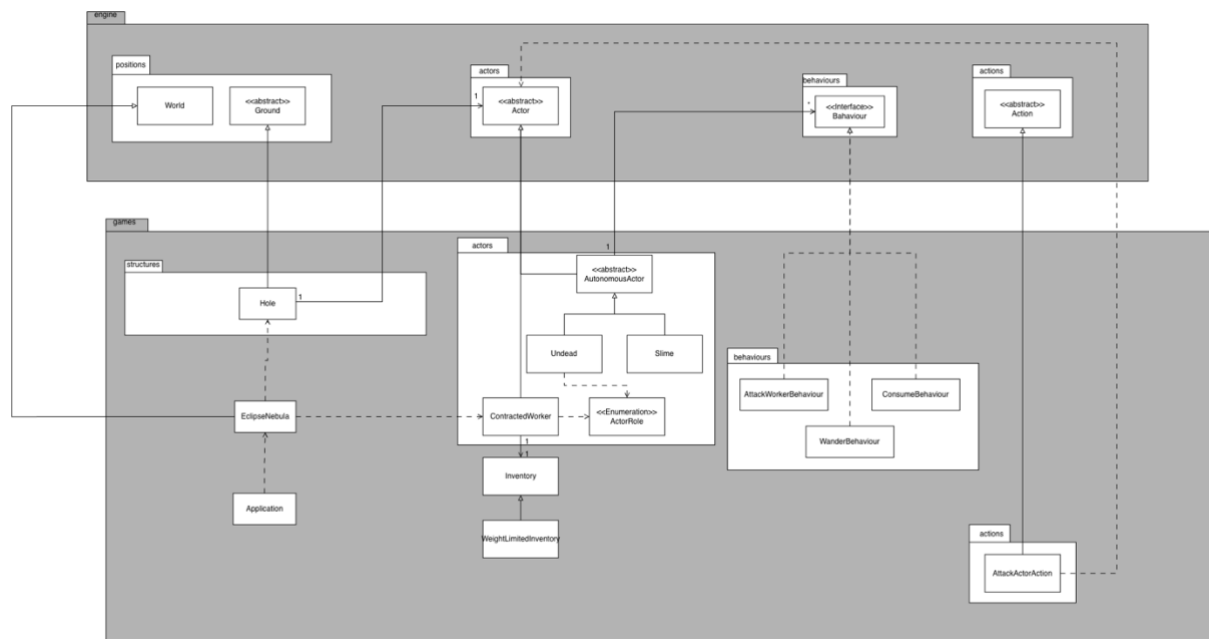
REQ 1 UML Class Diagram:



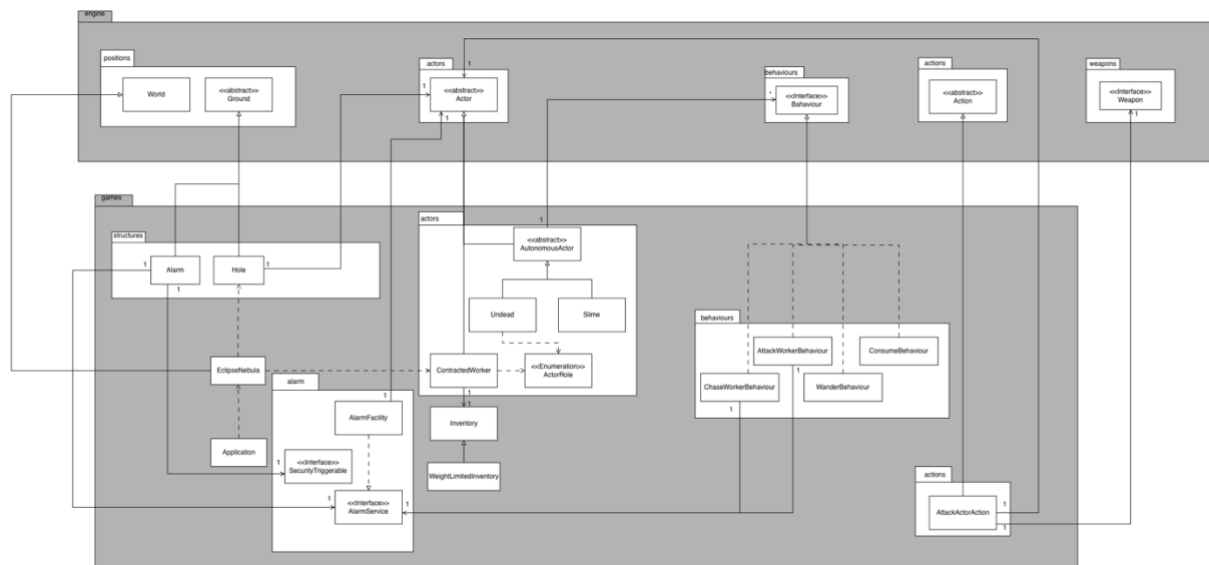
REQ 2 UML Class Diagram:



REQ 3 UML Class Diagram:



REQ 4 UML Class Diagram:



Design Rationale

Req 1

1. Refactor the codebase into modular, domain specific sub-packages.

Advantages	Disadvantages
High cohesion and logical grouping, strongly support the Single Responsibility (SRP) at an architectural level. Easier to navigate, debug and maintain. Encapsulates related logic, making dependencies clearer.	Requires strict management of package imports and careful use of access modifier.

2. Create a generic Consumable interface and a dedicated ConsumeAction controller. Modify Flask to implement Consumable.

Advantages	Disadvantages
High Extensibility (OCP): The engine can process any future consumable (Items or Ground tiles) simply by having them implement Consumable. Consume Action does not need to change. ConsumeAction is solely handles the transaction and memory management (removing item from the map/inventory). Flask solely manages its uses and effect. By adding an isConsumed() default method to the interface, the Flask easily overrides the to return false even when empty, satisfying the requirement that the empty flask remains in the inventory.	Requires abstracting the consumption logic away from the specific item classes.

3. Create a FirstAidKit class extends item class, a UseFirstAidKitAction class and leverage engine polymorphism for the cooldown timer when carried.

Advantages	Disadvantages
This follows the SRP as the cooldown responsibility stay in item and executive effect is in UseFirstAidKitAction. This also, minimal dependencies as no global scheduler needed, which keeps coupling low and concrete dependencies small.	Behaviour relies on understanding engine tick semantic for inventory items, hidden lifecycle coupling can weaken dependency clarity (DIP) If engine lifecycle is misunderstood, cooldown bugs can be subtle because timing policy is implicitly tied to framework callbacks (SRP boundary risk) Extending an abstract class introduces tight coupling between the concrete class and the abstract class, changes to the abstract class may result in modification of all its subclasses, increasing the need of code maintenance.

4. Create SterilisationBox class extends item class and Ability Enumeration with STERILISE ability.

Advantages	Disadvantages
SterilisationBox has one clear job: grant sterilise capability (SRP). New interactions can check ability instead of concrete class type (OCP). Other code depends on capability, not item internals (DIP).	Too many enum flags can become messy over time (ISP pressure). Extending an abstract class introduces tight coupling between the concrete class and the abstract class, changes to the abstract class may result in modification of all its subclasses, increasing the need of code maintenance.

5. Implement AccessCard class extends item class. The AccessCard class is simply a standard item with a weight of 1, scan adjacent exits and expose UnlockDoorAction for locked doors.

Advantages	Disadvantages
Capability is attached directly to the card, so possession naturally controls available actions and keeps responsibility with the granting object (SRP). Decouples door unlocking from worker class logic, letting high-level interaction depend on item abstraction rather than worker condition branches (DIP, OCP).	Item now knows local traversal context, which can blur responsibility boundaries if interaction logic grows too much Extending an abstract class introduces tight coupling between the concrete class and the abstract class, changes to the abstract class may result in modification of all its subclasses, increasing the need of code maintenance.

ConsumeAction can operate on any item implementing the interface. It also supports the **Open/Closed Principle**, allowing new consumable items to be added without modifying existing logic.

The Flask is designed as a multi-use consumable with an internal usage counter, ensuring it remains in the inventory even after depletion. This design promotes encapsulation, as the state and usage logic are managed within the class itself. The FirstAidKit introduces time-based behaviour through a cooldown mechanism, which progresses only when the item is carried. This responsibility is encapsulated within the item, ensuring adherence to the **Single Responsibility Principle** and avoiding unnecessary dependencies on the actor or external systems.

The AccessCard enables interaction with Door objects via a dedicated UnlockDoorAction, demonstrating separation of concerns between items and actions. Similarly, the SterilisationBox provides functionality that affects consumables, promoting **low coupling** by interacting through abstractions rather than concrete implementations.

To enforce game balance, key items such as the AccessCard, FirstAidKit, and SterilisationBox are instantiated only once within the game world. This design decision ensures shared responsibilities among players while avoiding duplication of critical resources. It also simplifies system management by preventing multiple instances of unique items.

Overall, the design effectively utilizes inheritance, interfaces, and composition to achieve high cohesion and low coupling. By adhering to SOLID principles, particularly the Single Responsibility Principle, Open/Closed Principle, and Dependency Inversion Principle, the system is modular, extensible, and maintainable, providing a strong foundation for future enhancements.

Req 2

1. Create simple concrete class Apple extending Item class, utilizing the pre-built ItemStatistics.WEIGHT capability and implement the Consumable Interface.

Advantages	Disadvantages
This design promotes code reuse because it extends the existing Item class, which already provides common item behaviour. It follows the Single Responsibility Principle because the Apple class only needs to define its own consumption behaviour and weight. It also supports polymorphism because a shared ConsumeAction can work with any object that implements Consumable. In addition, it satisfies the Open/Closed Principle because new consumable items can be added without changing existing action logic.	This approach increases the number of small classes in the system, which may add some maintenance overhead. Limited inheritance to only 1 abstract class per concrete class, limits the development of functionality of each individual concrete class. Extending an abstract class introduces tight coupling between the concrete class and the abstract class, changes to the abstract class may result in modification of all its subclasses, increasing the need of code maintenance.

2. Create simple concrete class Cookies extending Item class, utilizing the pre-built ItemStatistics.WEIGHT capability and implement the Consumable Interface.

Advantages	Disadvantages
This design demonstrates strong encapsulation because the Cookies class manages its own remaining uses internally. It follows the Single Responsibility Principle because the class is only responsible for its own state and consumption logic. It also remains Open/Closed Principle compliant because no existing action logic needs to change when adding similar multi-use items. In addition, it has high cohesion because all cookie-related logic is kept inside one class.	Limited inheritance to only 1 abstract class per concrete class, limits the development of functionality of each individual concrete class. Extending an abstract class introduces tight coupling between the concrete class and the abstract class, changes to the abstract class may result in modification of all its subclasses, increasing the need of code maintenance.

3. Create Lantern and Fire classes and distribute responsibilities using the engine's polymorphic tick methods. The Lantern controls its own leaking, and Fire controls its own burning.

Advantages	Disadvantages
This design strongly follows the Single Responsibility Principle because Lantern handles leaking and Fire handles burning. It also provides good encapsulation because each class controls its own lifecycle and behaviour. The design supports polymorphism through the engine's tick() method, which allows different objects to define their own time-based logic. It also satisfies the Open/Closed Principle because new ticking behaviour can be added without modifying existing classes.	Behaviour is spread across multiple tick() methods, so the control flow is less centralised and harder to follow. This can make debugging more difficult because the logic happens indirectly over time.

4. Create encapsulated status classes (Poison and Burn) located in the game.statuses package. These objects are attached to the affected actor and handle their own lifecycle.

Advantages	Disadvantages
This design provides excellent separation of concerns because each status class manages only one effect. It follows the Single Responsibility Principle because Poison and Burn each handle their own duration, damage, and removal logic. It is also highly reusable because the same status effects can be attached to different actors. The design supports the Open/Closed Principle because new status effects can be added without changing the Actor class.	This approach increases the number of classes and runtime objects in the system. The effect is applied indirectly through attached status objects, which can make the logic harder to read at first.

5. Create simple concrete class `FloppyDisk` extending `Item` class, utilizing the pre-built `ItemStatistics.WEIGHT` capability.

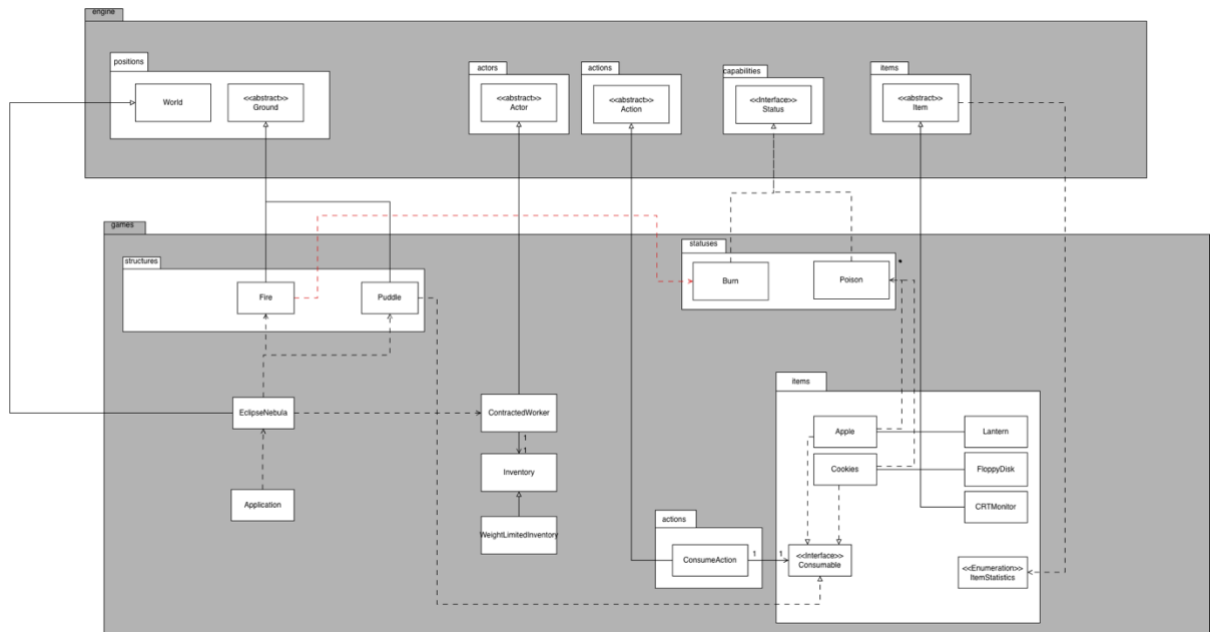
Advantages	Disadvantages
This design has high cohesion because <code>FloppyDisk</code> is only responsible for defining its own weight. It follows the DRY principle by reusing the built-in weight capability instead of creating custom weight logic. It also supports the Open/Closed Principle because new weighted items can be added without modifying the inventory system. The design keeps coupling low because other systems depend on the capability rather than the concrete item type.	Extending an abstract class introduces tight coupling between the concrete class and the abstract class, changes to the abstract class may result in modification of all its subclasses, increasing the need of code maintenance.

6. Create simple concrete class `CRTMonitor` extending `Item` class, utilizing the pre-built `ItemStatistics.WEIGHT` capability.

Advantages	Disadvantages
This design has high cohesion because <code>CRTMonitor</code> is only responsible for defining its own weight. It follows the DRY principle by reusing the built-in weight capability instead of creating custom weight logic. It also supports the Open/Closed Principle because new weighted items can be added without modifying the inventory system. The design keeps coupling low because other systems depend on the capability rather than the concrete item type.	Extending an abstract class introduces tight coupling between the concrete class and the abstract class, changes to the abstract class may result in modification of all its subclasses, increasing the need of code maintenance.

7. Create simple concrete class Puddles extending Ground class, and implement the Consumable Interface.

Advantages	Disadvantages
This design supports strong polymorphism because Puddles can be treated the same way as other consumable objects. It follows the DRY principle by reusing the existing ConsumeAction instead of writing separate drink logic. It also satisfies the Open/Closed Principle because new environmental consumables can be introduced without changing the core action system. The design is encapsulated because the Puddle class defines its own consumption behaviour and can represent an infinite source by overriding isConsumed().	This design may be conceptually confusing because a Ground object behaving like a consumable is not the most intuitive model. It stretches the meaning of the Consumable interface, which may reduce clarity for future developers. Without clear documentation, this could make the code harder to understand and maintain



The diagram above shows the final design of Requirement 2, which extends the system with consumable items, environmental interactions, and modular status effects within the “99-Deprecated” moon.

Consumable items such as Apple and Cookies are implemented as concrete classes extending the abstract Item class while also implementing the Consumable interface. This design allows all consumable objects to share a common contract while defining their own unique effects. By using a shared ConsumeAction, the system achieves **code reuse (DRY principle)** and polymorphism, as the same action can be applied to any consumable. The consumption logic is strictly encapsulated within each item, ensuring adherence to the **Single Responsibility Principle (SRP)**, while also satisfying the **Open/Closed Principle (OCP)**, as new consumables can be added in the future without modifying existing action logic.

The interaction between consumables and the SterilisationBox is handled through capability checks (Ability.STERILISE) rather than hardcoded conditional statements. This promotes **low coupling and high flexibility**, ensuring that both inventory items and environmental consumables can respond dynamically based on the actor’s state without introducing direct dependencies between concrete classes.

The Puddle class extends the abstract Ground class and implements the Consumable interface, allowing environmental objects to be treated uniformly with inventory-based consumables. This demonstrates strong polymorphism and the reuse of existing actions, as the same ConsumeAction can be executed when a worker is standing on a puddle. By encapsulating its own logic for poisoning or healing, the Puddle class maintains **high cohesion and adheres to SRP**. This approach **also satisfies OCP**, as

new environmental consumables can be introduced without modifying existing engine systems.

The Lantern and Fire classes utilise the engine's polymorphic tick() mechanism to implement autonomous behaviour. The Lantern autonomously manages its own fuel and probabilistic leaking, while the Fire manages its own duration and damage effects on actors. This separation ensures strong encapsulation and adherence to **SRP**, as each class is responsible only for its own lifecycle. By distributing behaviour across objects rather than centralising it in the actor's turn loop, the design drastically **reduces coupling and improves extensibility**.

The introduction of status effect classes such as Poison and Burn further enhances modularity. These classes encapsulate their own duration and damage logic and can be dynamically attached to actors. This design promotes **high cohesion** and successfully avoids bloating the Actor class, preventing it from becoming a monolithic "God Class." It also **satisfies OCP**, as new status effects can be added without modifying existing actor implementations, fully supporting reusability across different game entities.

Finally, items such as FloppyDisk and CRTMonitor utilise the pre-existing ItemStatistics.WEIGHT capability to define their weight. This demonstrates adherence to the **DRY principle**, as the engine's weight system is reused rather than reimplemented. It also **ensures low coupling**, since inventory capacity behaviour depends entirely on generic capabilities rather than checking for concrete item types.

Overall, the design heavily leverages abstraction, interfaces, and composition to ensure high cohesion and low coupling across the system. By strictly adhering to SOLID principles, particularly SRP, OCP, and the Dependency Inversion Principle, the system is flexible, extensible, and highly maintainable.

Req 3

1. Create a concrete class `Hole` extends the `Ground` class. It accept a list of factory methods or `Supplier<Actor>` into the `Hole` constructor. The `Hole` uses a random number generator to select one of the provided suppliers when the 20-turn spawn interval is reached.

Advantages	Disadvantages
This design follows the Single Responsibility Principle, as the <code>Hole</code> class is only responsible for managing spawn timing and selection, not knowing what specific enemies are created. It demonstrates Dependency Inversion Principle, because the <code>Hole</code> depends on abstractions (<code>Supplier<Actor></code>) rather than concrete enemy classes. The design also satisfies the Open/Closed Principle, as new enemy types can be introduced without modifying the <code>Hole</code> class. Additionally, it promotes high scalability and flexibility, since different maps can pass different suppliers to spawn different enemies	However, this design increases the complexity of system setup, as suppliers or factory methods must be correctly configured and passed into the constructor

2. Create an abstract class `autonomousActor` extends `Actor` class that holds a list of abstract Behaviour objects. It universally overrides `playTurn()` to simply iterate through the list until a valid action is found.

Advantages	Disadvantages
This design promotes code reuse (DRY principle) by extracting common auto decision-making logic into a single base class. It follows the Open/Closed Principle, as new behaviours can be added without modifying the base class logic. It also supports polymorphism, since different behaviours can be applied dynamically through a common interface. Furthermore, it reduces duplication across enemy classes, improving maintainability and scalability.	However, this design requires careful management of behaviour priorities or ordering, as incorrect ordering may lead to unintended AI decisions. It also introduces abstraction overhead, which may make the system harder to understand for new developers Extending an abstract class introduces tight coupling between the concrete class and the abstract class, changes to the abstract class may result in modification of all its subclasses, increasing the need of code maintenance.

--	--

3. Create highly modular, isolated classes (WanderBehaviour, AttackWorkerBehaviour) implementing the engine's Behaviour interface. These classes calculate logic and return an Action.

Advantages	Disadvantages
This design strongly follows the Single Responsibility Principle, as each behaviour class is responsible for a single type of logic (e.g., wandering or attacking). It promotes high cohesion, since related logic is contained within one class. The design is highly reusable, as multiple actors can share the same behaviour classes. It also satisfies the Open/Closed Principle, allowing new behaviours to be added without modifying existing actor classes.	The main drawback is the increase in the number of classes, which may make the project structure more complex. Additionally, understanding the full behaviour of an actor requires looking across multiple files, which may reduce code readability

4. Create ConsumeBehaviour class implementing the engine's Behaviour interface rely entirely on the polymorphic Consumable interface established in Req 1. It checks the location for items/grounds implementing this interface.

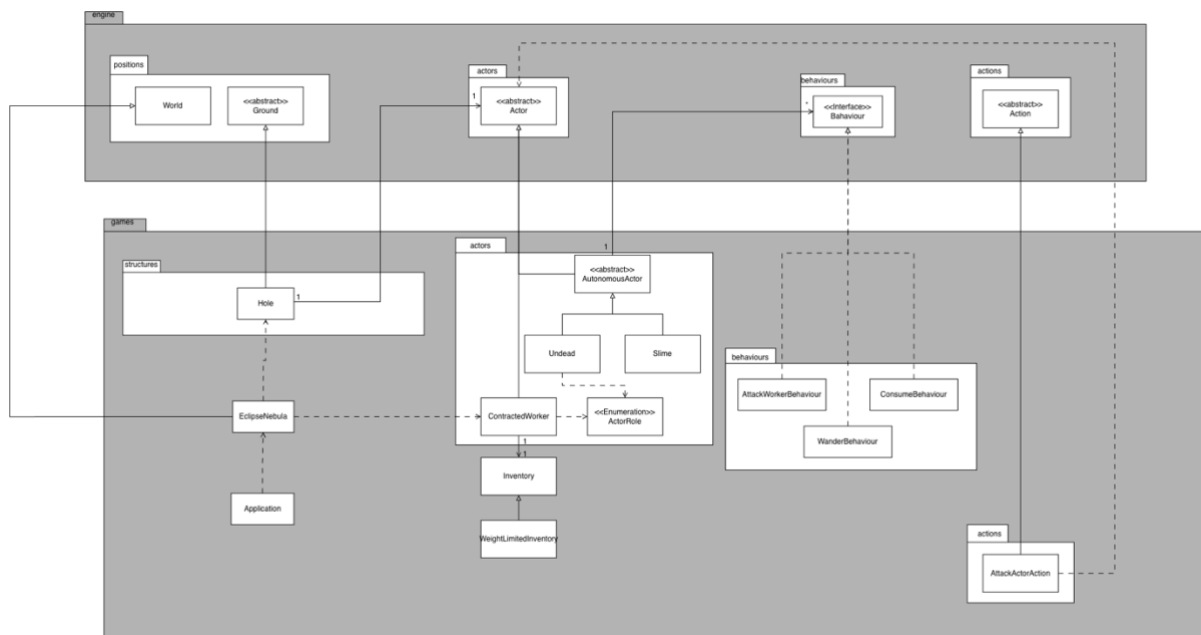
Advantages	Disadvantages
This achieve perfect decoupling as the Slime does not know or care what it is eating, only that the item satisfies the abstract Consumable contract. Because the interface defines the consumeBy(Actor) method natively inside the item, the Slime naturally suffers the exact same poison/healing effects as the player without rewriting any item logic specifically for enemies.	Requires ensuring that the item's consumeBy effects are programmed generically enough to be applied to any Actor parameter, not just the player character.

5. Create concrete Undead class extends AutonomousActor class. The class contain almost no logic, the constructors merely register their stats (e.g., 15 HP for Undead) and add instances of specific behaviors to the list.

Advantages	Disadvantages
This design results in highly clean and maintainable code, as the class only configures its own properties and behaviours. It follows the Single Responsibility Principle, since the class only defines configuration (e.g., health and behaviours). It promotes low coupling, as logic is delegated to behaviour classes. The design is also highly extensible, as behaviours can be changed without modifying the class itself, satisfying the Open/Closed Principle.	Extending an abstract class introduces tight coupling between the concrete class and the abstract class, changes to the abstract class may result in modification of all its subclasses, increasing the need of code maintenance.

6. Create concrete Slime class extends AutonomousActor class. The class contain almost no logic, the constructors merely register their stats (e.g., 25 HP for Slime) and add instances of specific behaviors to the list.

Advantages	Disadvantages
This design follows the same clean architecture principles as Undead, ensuring consistency across enemy implementations. It adheres to the Single Responsibility Principle, as the class only defines its own configuration and does not manage behaviour logic. It supports polymorphism and reuse, since behaviours can be shared across different actors. It also satisfies the Open/Closed Principle, allowing behaviour changes without modifying the class.	Extending an abstract class introduces tight coupling between the concrete class and the abstract class, changes to the abstract class may result in modification of all its subclasses, increasing the need of code maintenance.



The diagram above shows the final design of requirement 3, which utilizes the chosen designs of the parts to implement modular entity , autonomous behaviors, and dynamic environmental spawning.

The new concrete classes Slime and Undead extend the abstract AutonomousActor class, which in turn extends the engine's core Actor class. Since we want both hostile entities (like the Undead) and passive entities (like the Slime) to share the trait of autonomously executing actions each turn without player input, it is logical to abstract this shared turn-processing logic into the AutonomousActor class to **avoid massive repetitions in code (DRY)**.

Using individual, encapsulated classes (WanderBehaviour, AttackWorkerBehaviour, ConsumeBehaviour) that implement the Behaviour interface allows us to separate different auto decision-making processes into their own isolated files. This keeps the core actor classes from becoming bloated "God Classes" and results in much better maintainability of the system (**Single Responsibility Principle**). This approach also allows **for easy extensions** to the game's auto character; we can give an enemy entirely new logic simply by passing a new behavior into its priority list, rather than hardcoding new conditional branches.

Furthermore, the Hole class's design relies on a generic Supplier<Actor> list rather than hardcoded dependencies on the concrete Slime or Undead classes. This implementation allows us to remove multiple dependencies of the environmental spawner on specific enemy types. When introducing new creatures to the game in the

future, we do not need to modify the Hole class source code to spawn them, as the class acts purely as a generic factory executor (**Open-Closed Principle**).

Overall, the design effectively utilizes inheritance, interfaces, and composition to achieve high cohesion and low coupling. By having Slime and Undead extend the abstract AutonomousActor class, the system leverages inheritance to centralize and reuse the core logic for processing AI-driven turns. The implementation of the Behaviour interface, coupled with composition, allows specific AI strategies, such as WanderBehaviour, AttackWorkerBehaviour, and ConsumeBehaviour to be modularly assigned to entities without bloating the core actor classes. This architecture strictly adheres to the Single Responsibility Principle and the Open/Closed Principle, providing a scalable foundation where new creature types or complex behavioral patterns can be introduced without modifying existing actor or engine code.

Req 4

1. Create an AlarmFacility class that implements an AlarmService interface. This service acts as the single source of truth for the timer and target and is passed as a dependency to the entities that require it.

Advantages	Disadvantages
This design adheres to SRP as the AlarmFacility has exactly one responsibility: ticking the countdown timer and holding the active state. It delegates all other logic (rendering, collision) to other classes. This design also adheres to DIP as entities like Door and Undead depend entirely on the abstract AlarmService interface. The concrete implementation of the alarm facility can be swapped or rewritten without breaking any dependent code.	Injecting dependencies (passing the AlarmService via constructors) slightly increases the boilerplate setup code during map generation.

2. Create a generic SecurityTriggerable interface with a triggerAlarm() method. The ContractedWorker implements this interface. The Alarm ground tile only checks if the actor implements this interface via the engine's Location.getActorAs() method.

Advantages	Disadvantages
This design adheres to ISP as the Alarm ground tile now depends only on an interface with a single method. It is completely blind to the rest of the Actor's heavy data. This design adheres to OCP as the alarm trigger system is completely closed for modification but open for extension. Any future entity can implement SecurityTriggerable to interact with alarms without modifying the Alarm ground tile at all.	Requires the creation of an additional interface, slightly expanding the project's package footprint.

3. Create an Alarm tile extending the Ground class that acts purely as a physical sensor. When an actor steps on it, the tile checks if they implement a specific interface and executes `securityTriggerable.triggerAlarm(alarmService)`.

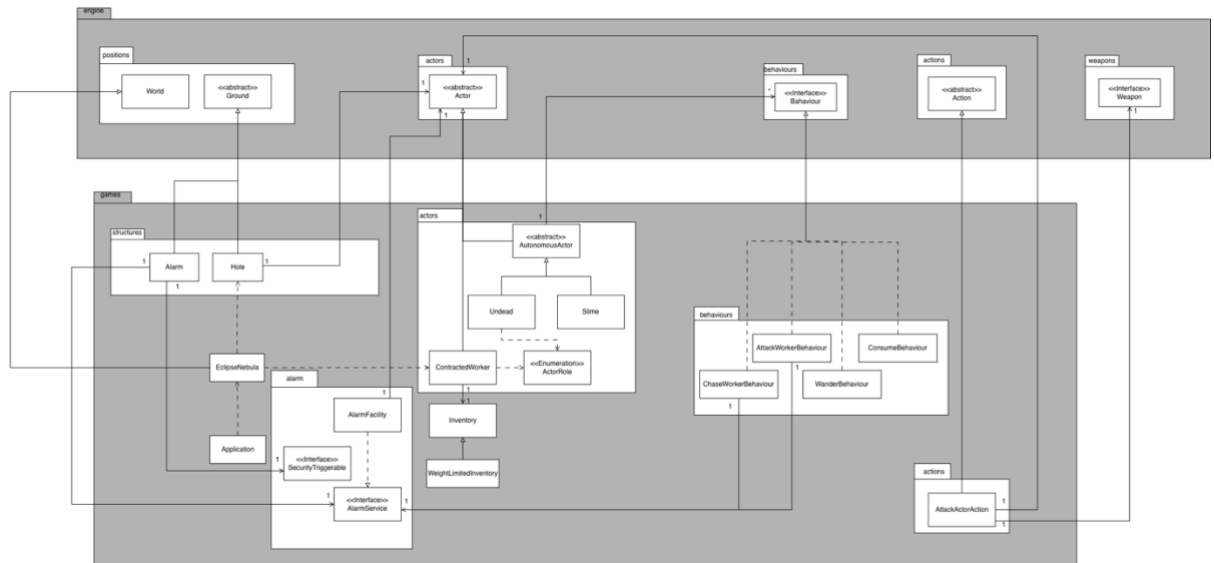
Advantages	Disadvantages
This design adheres to SRP and perfect cohesion. The ground tile has exactly one job, detecting physical collisions (when an actor steps on it). It blindly delegates the actual punishment or duration logic to the actor, keeping game-state business logic out of environmental assets. This adheres to OCP (Open/Closed Principle) as the environment tile is completely "blind" to <i>who</i> is stepping on it. If a new entity (like an allied drone or a hostile intruder) is added in a future update that also needs to trigger alarms, they simply implement the interface. The Alarm class code never has to be modified Adheres to ISP (Interface Segregation Principle): By querying the actor's capability via <code>Location.getActorAs(SecurityTriggerable.class)</code>, the ground tile only depends on a micro-contract with a single method. It successfully avoids being tightly coupled to the heavy data and methods contained within the base Actor class.	This design requires the creation and maintenance of an additional micro-interface (<code>SecurityTriggerable</code>) strictly for this interaction, slightly expanding the project's package size.

4. The Door class queries the injected `AlarmService` dynamically.
`Door.canActorEnter()` automatically returns false, and `UnlockDoorAction` immediately aborts returning a failure message if `alarmService.isActive()` is true.

Advantages	Disadvantages
This design adheres to SRP as it perfect decentralization. The <code>AlarmService</code> only broadcasts its state, the Door manages its own localized rules for entry based on that state. This design adheres to OCP as new consequences can be added simply by creating new items/structures that read the <code>AlarmService</code> state. The core alarm code never needs to be touched to add new punishments.	Doors require a persistent, injected reference to the <code>AlarmService</code> to check its status during their internal logic checks.

5. Create a dedicated ChaseWorkerBehaviour utilizing a Breadth-First Search (BFS) algorithm to calculate the true shortest path to the triggeringWorker (provided by the AlarmService). This behavior is injected as the highest priority in the Undead's behaviour list.

Advantages	Disadvantages
This design adheres to SRP as the complex math for tracking, visiting locations, and finding the shortest path to a target is perfectly isolated inside a single behaviour class. This design follow OCP/LSP. By utilizing polymorphic behaviours as interchangeable strategies, you can dynamically give any entity the ability to chase the player simply by adding the behaviour to its list, without modifying any actor source code.	Implementing advanced pathfinding (BFS) in an isolated behaviour class requires handling edge cases (like locked doors blocking the path) via the abstract Location and Exit objects.



The diagram above shows the final design of requirement 4, which utilizes the chosen designs of the aforementioned parts to implement a globally tracked facility alarm, dynamic environmental triggers, and automated consequence systems (lockdowns and chasing).

The new concrete class AlarmFacility implements the AlarmService interface to act as the single source of truth for the game's lockdown state. To trigger this state, the ContractedWorker implements the SecurityTriggerable interface. By having the Alarm ground tile rely strictly on this interface rather than checking for a specific Actor class, we ensure the ground tile only depends on the exact method it needs (triggerAlarm). This keeps the dependency footprint as small as possible (**Interface Segregation Principle**) and prevents environmental tiles from dictating actor-specific rules.

Using a dedicated ChaseWorkerBehaviour to handle the complex Breadth-First Search (BFS) pathfinding allows us to separate the intensive routing mathematics from the Undead's core data structure (**Single Responsibility Principle**). Because it implements the standard Behaviour interface, the Undead can seamlessly swap to this aggressive strategy the moment the alarm sounds without requiring complex internal logic rewrites.

Crucially, high-level map structures like the Door and behaviors like ChaseWorkerBehaviour have a strict dependency on the abstract AlarmService interface, rather than the concrete AlarmFacility class. This implementation allows us to completely decouple the map's reactive elements from the underlying time-tracking mechanics. When introducing new consequences for the alarm in the future (such as spawning security drones), these new entities

can safely query the abstraction without risking modifications to the core engine loop (Open-Closed Principle and Dependency Inversion Principle).

Overall, the design effectively utilizes abstraction, interfaces, and dependency injection to achieve a highly decoupled and responsive facility system. By centralizing the global lockdown state within the AlarmService interface and its AlarmFacility implementation, the system adheres strictly to the Single Responsibility Principle and the Dependency Inversion Principle, ensuring that high-level map structures depend on abstractions rather than concrete implementations. The use of the SecurityTriggerable micro-interface for alarm activation successfully demonstrates the Interface Segregation Principle, allowing environmental tiles like the Alarm to interact with actors through a minimal, specific contract. Furthermore, by isolating advanced pathfinding logic within the ChaseWorkerBehaviour, the design ensures that complex consequences are modular and extensible. This approach provides a modular foundation that seamlessly supports the addition of future security features or environmental triggers.